How Much Headroom Do Smoothing Schedules Have? A Noise-Calibrated Study of LLM-Evolved Schedules in Differentiable VLSI Placement

Chaithu Talasila
Independent Researcher
github.com/themoddedcube/evoplace

Abstract—Large language models (LLMs) have recently been used to evolve algorithmic components of GPU-accelerated analytical placers, with reported wirelength improvements of 5–17% on macro-heavy benchmarks. We investigate a component class untouched by prior work: the smoothness (γ) schedule of the weighted-average wirelength model in DREAMPlace. We present EvoPlace[1], an evolution harness built on three components: (i) a hook-injection layer that lets a candidate schedule—a pure Python function—steer an unmodified DREAMPlace build; (ii) a cascade evaluator with stage-matched baselines that rejects poor candidates at 13× lower cost than a full placement; and (iii) a noise-calibrated evaluation protocol that measures the single-seed fitness noise floor ($\sigma \approx 0.15\%$ normalized HPWL), gates every campaign on hook-liveness probes, and confirms final rankings with paired multi-seed re-evaluation. Across a 200-iteration evolution campaign on ISPD 2015 standard-cell designs, the search produced exactly one statistically real improvement: $+0.315\% \pm 0.09\%$ (SEM) HPWL over the tuned DREAMPlace default—and the single-seed ranking inverted under multi-seed re-evaluation, with the apparent best candidate revealed as pure seed luck. We conclude that the γ-schedule headroom on converged standard-cell placement is bounded near 0.3%, an order of magnitude below the single-seed "improvements" that an uncalibrated search happily reports. Our protocol generalizes to any stochastic-EDA search loop, and our negative result quantitatively corroborates component-importance orderings reported in concurrent work.

I. Introduction

Analytical global placement casts cell spreading as continuous optimization: DREAMPlace [2] expresses the ePlace electrostatic formulation [3] in a deep-learning toolkit and achieves order-of-magnitude speedups on GPUs. The objective combines a smooth approximation of half-perimeter wirelength (HPWL) with an electrostatic density penalty, minimized by Nesterov's method. Two scalar schedules steer this optimization: the weighted-average wirelength smoothness parameter γ, which trades gradient smoothness against HPWL fidelity, and the density weight λ. Both are hand-designed heuristics, tuned by experts over years.

A natural question follows from the recent success of LLM-guided algorithm evolution [4]: can an LLM discover a better schedule? Concurrent work has evolved other DREAMPlace components —initialization, preconditioner, and optimizer—reporting case-by-case HPWL reductions of 5–8% on average [1]. Schedules, however, were not examined, and the reliability of small reported gains in this field is clouded by a methodological hazard: placement is stochastic, single-seed evaluation is near-universal, and few studies report variance at all [5], [6].

This paper reports a complete, noise-calibrated campaign of LLM-guided γ-schedule evolution. Our contributions are:

- A measurement-first evolution harness. Schedule candidates are pure Python functions injected into an unmodified DREAMPlace build through a hook layer; the evaluation harness is frozen for the life of the campaign; and a cascade evaluator with stage-matched baselines rejects 93% of candidates at a fraction of full-evaluation cost.
- A noise-calibrated evaluation protocol. We measure the single-seed fitness noise floor, gate the campaign on differential hook-liveness probes, and confirm all final claims by paired multi-seed re-evaluation against the default schedule on identical (benchmark, seed) pairs.
- A quantitative boundary result. Across 200 evolution iterations, the best schedule that survives multi-seed scrutiny improves HPWL by $0.315\% \pm 0.09\%$; the single-seed winner is revealed as noise. We conclude that γ-schedule headroom on converged standard-cell placement is bounded near 0.3%.

We deliberately report a bounded-headroom (negative) result with full provenance, including a record-keeping artifact we caught and corrected. In a field where almost no small improvement is variance-qualified, we believe the calibration methodology is as much a contribution as the

[1] Our system was developed under this name prior to our awareness of the concurrent, independent system of Yao et al. [1] bearing the same name; we retain it for repository consistency and cite their work explicitly throughout.

bound itself.

II. Related Work

GPU-analytical placement. DREAMPlace [2] implements ePlace/RePlAce-style electrostatic placement with CUDA kernels under a PyTorch-like API; successive versions added timing-driven net weighting and mixed-size support. Its γ schedule follows an overflow-driven exponential heuristic; our seed program reproduces this default to machine precision so that evolution starts from parity.

LLM-evolved placement components. Yao et al. [1] evolve initialization, preconditioner, and optimizer implementations with GPT-4o at scale ($\geq$1000 candidates per component, $\sim$210 GPUs), reporting 5.05–8.30% average case-by-case HPWL reductions, with initialization dominating and an order-of-magnitude collapse when one evolved algorithm must generalize across designs. Their clean standard-cell-like cases improve by only 0.03–0.59%—consistent with the bound we measure for schedules. VeoPlace [7] evolves macro placements with vision-language-model feedback and is one of the few systems in this space to report multi-seed means with standard errors.

Evaluation reliability in ML-for-EDA. The reassessments of RL-based macro placement [8] by Cheng et al. [5] and Markov [6] demonstrated that headline gains can be artifacts of undisclosed initialization and weak baselines. RoutePlacer [9] is a rare example reporting mean±std over five seeds. Our protocol extends this thread: we measure the noise floor first and require any claimed improvement to clear it under paired re-evaluation.

III. The EvoPlace Harness

A. Hook injection

A patched PlaceObj consults a module-level hook registry at every global-placement iteration. When a candidate registers gamma_schedule(iteration, total_iterations, overflow, hpwl_history), its return value (clamped to a documented range) overrides the built-in heuristic; when no hook is set, DREAMPlace behaves identically to upstream. Candidates are therefore pure Python functions, loaded dynamically—no rebuild, no source swap. The same registry exposes λ-schedule, initialization, and timing-loss slots.

B. Frozen evaluator and cascade

A fixed harness (evaluator/run_placement.py) builds DREAMPlace parameters from per-benchmark JSON, injects hooks, runs placement to convergence (stop overflow 0.07, legalization included), and extracts final HPWL and overflow. The file is never modified during a campaign, preventing fitness-definition drift. Candidate evaluation uses a three-stage cascade (50, 300, 2000 iterations) with rejection thresholds (2.0, 1.3) normalized against stage-matched baselines: a truncated run of the default schedule lands 2–2.5$\times$ above its converged HPWL, so normalizing truncated candidates against converged baselines would cull everything, including the default itself. Stage-0 rejection costs $\sim$5 s versus $\sim$30 s for a full pass on our hardware.

C. Evolution loop

We drive evolution with an OpenEvolve-style loop: MAP-Elites over (normalized HPWL, convergence speed) with four islands of 25 programs, diff-based mutations from a Claude model ensemble (temperature 0.8), and TSV logging of every candidate with code hashes. A lightweight hill-climbing fallback loop shares the same evaluator.

IV. Noise-Calibrated Evaluation Protocol

Noise floor. Repeated full placements of the same schedule under different seeds establish a single-seed fitness standard deviation of $\sigma \approx 0.15\%$ normalized HPWL on our benchmarks. Every single-seed score in the campaign is interpreted against this floor; we pre-registered the rule that no single-seed improvement below $3\sigma \approx 0.45\%$ may be treated as a result.

Liveness gating. An earlier CPU-era campaign of ours silently evaluated vanilla DREAMPlace for every candidate—the hooks were not wired into the installed tree—producing 20 near-identical scores whose spread was pure numerical noise. The failure motivated two standing gates run before any campaign: (i) the seed program must reproduce the default baseline at norm. HPWL $= 1.0 \pm 0.01$ through the real cascade; and (ii) a differential probe—a deliberately degenerate schedule (constant $\gamma = 0.01$)—must produce a markedly different outcome (in practice, cascade rejection). Together these prove the candidate code path actually steers the placement.

Paired multi-seed confirmation. After a campaign, top candidates and the seed program are re-evaluated with full placements on a grid of (benchmark, seed) pairs; each run is normalized by the default schedule's HPWL on the identical pair. Pairing cancels per-seed placement luck. The seed program rides along as a calibration row whose paired ratio must be $\approx$1.000; deviation indicates a broken protocol rather than a discovery.

V. Experimental Setup

Hardware. All experiments ran on an NVIDIA DGX Spark: a GB10 Grace–Blackwell system (aarch64, compute capability 12.1, 20 cores, 121 GiB unified memory), CUDA 13.0, PyTorch 2.12/cu130. The full project test suite (114 tests) passes on this platform.

Benchmarks and baselines. We evolve on fft_1 and confirm on fft_1/fft_2 from the ISPD 2015 contest suite [10]. Baselines are re-measured on the target hardware (Table I); converged HPWL agrees with our reference RTX 3060 measurements within 0.1–1.6% while running $\sim$5$\times$ faster.

TABLE I
Default-schedule baselines (seed 42, converged, legalized).

	GB10 (this work)		RTX 3060 (ref.)	
Design	HPWL	time	HPWL	time
fft_1	2.183×10^6	5.5 s	2.180×10^6	29.4 s
fft_2	1.951×10^6	2.2 s	1.921×10^6	12.2 s

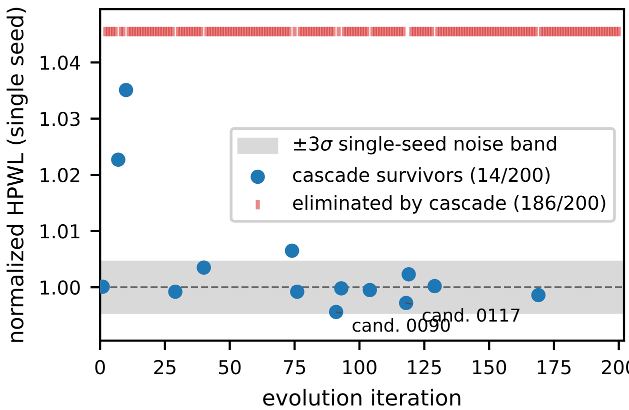

Fig. 1. Single-seed scores of all 200 evolution candidates. Survivor scores cluster inside the measured $\pm 3\sigma$ noise band; the cascade rejects 93% of candidates cheaply (top ticks).

Campaign. 200 evolution iterations, cascade evaluation throughout, ~90 minutes wall-clock including LLM mutation latency. Liveness gates passed before launch (seed at 0.99932; degenerate probe rejected at stage 1).

VI. Results

A. Evolution dynamics

Figure 1 shows every candidate score. The cascade rejected 186 of 200 candidates (93%), the majority at stage 0 or 1; the 14 survivors span 0.9956–1.0351. Two observations frame the analysis. First, the search does measure real differences: candidates at +2.3% and +3.5% are confidently separated from the baseline, and degenerate schedules are rejected outright. Second, every apparent improvement sits inside or near the $\pm 3\sigma$ single-seed noise band—the regime where ranking is unreliable by construction.

B. Multi-seed confirmation and the rank inversion

Table II and Figure 2 give the paired multi-seed outcome (5 seeds $\times$ 2 designs, 10 paired ratios per program). The calibration row behaves: the seed program lands at 1.00057 ± 0.0074, statistically indistinguishable from the default it re-implements. The single-seed winner (candidate 0090, single-seed 0.99563) regresses to exactly the baseline (1.00014 ± 0.0085): its -0.44% was seed luck. The single-seed runner-up (candidate 0117, single-seed 0.99716) is the only real improvement: 0.99685 ± 0.0027, i.e. +0.315% with a 95% CI excluding zero (≈3.7× SEM).

TABLE II
Paired multi-seed re-ranking (10 ratios per program; ratio < 1 improves on the default schedule at identical (design, seed)).

Program	Single-seed	Paired mean ± std	Verdict
cand. 0117	0.99716	0.99685 ± 0.0027	real, +0.315%
cand. 0090	0.99563	1.00014 ± 0.0085	noise
seed (default)	1.00010	1.00057 ± 0.0074	calibration ✓
neg. control A	—	1.02244 ± 0.0059	worse (rejected)
neg. control B	—	1.06626 ± 0.0046	worse (rejected)
neg. control C	—	1.07746 ± 0.0123	worse (rejected)

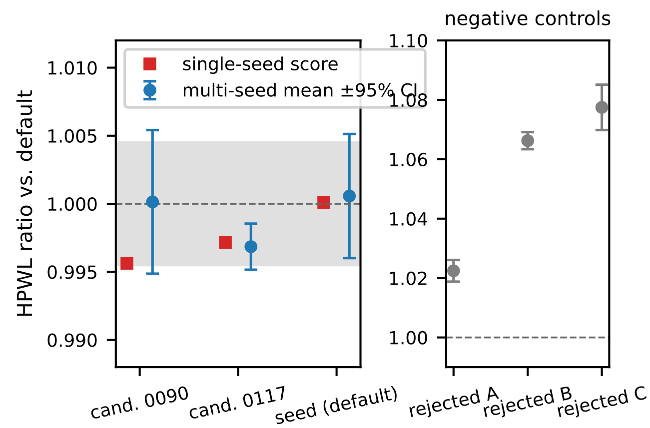

Fig. 2. Single-seed score (squares) vs. paired multi-seed mean ±95% CI (circles). The single-seed ranking inverts: the apparent winner (0090) regresses to baseline; the runner-up (0117) is the real improvement. Right: negative controls.

Three additional programs included through a record-keeping artifact (duplicate iteration indices in the result log caused stale selections; they correspond to cascade-rejected programs) serve as negative controls: the protocol measures them 2.2–7.7% worse with tight intervals, confirming sensitivity in both directions.

C. What the surviving schedule does

The confirmed schedule (Figure 3) replaces the built-in overflow-exponential heuristic with a blend of a cosine-warped geometric decay and overflow-coupled terms, with a floor that prevents deep annealing until both schedule progress and density relaxation have occurred. Qualitatively it anneals γ later and more smoothly than the default. The effect, however measurable, is small: +0.315% HPWL on designs where the default schedule has been tuned for years.

D. An accidental third measurement

After the campaign, visualization runs re-executed candidate 0117 and the default on fresh seeds; the evolved schedule won by +0.18% and +0.27% respectively—both within one standard error of the confirmed +0.315% estimate, an incidental out-of-protocol replication.

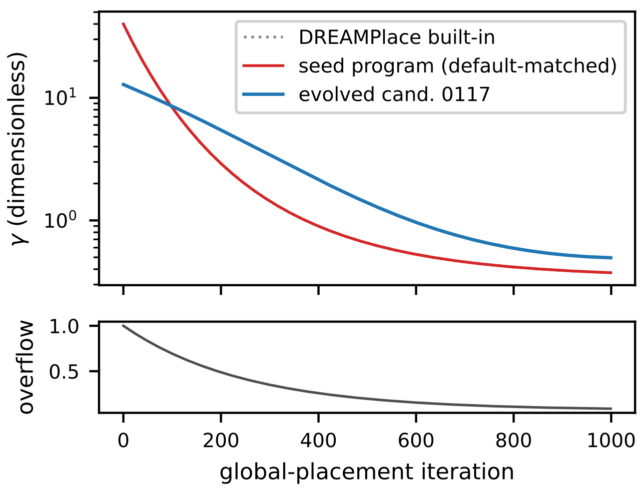

Fig. 3. γ trajectories under an illustrative overflow relaxation (bottom): DREAMPlace's built-in heuristic, the default-matched seed program, and the confirmed evolved schedule (cand. 0117).

VII. Discussion

Why the headroom is small. Three compounding causes. (i) The default schedule is already overflow-adaptive and extensively tuned; the remaining gain from trajectory shaping of an optimizer that converges to the same basin is intrinsically limited. (ii) Standard-cell ISPD 2015 designs present smooth placement landscapes; the literature's large evolved-component gains arise from macro handling on macro-heavy designs [1], a lever these benchmarks barely expose. (iii) Effect sizes ($\leq 0.3\%$) sit below the single-seed noise floor ($\sigma \approx 0.15\%$, so single observations are dominated by luck)—an evolutionary search cannot reliably climb a gradient its fitness oracle cannot resolve, and our rank inversion shows the consequence directly.

The single-seed hazard is general. Our campaign internally reproduced, at small scale, the pattern documented in the RL-placement reassessments [5], [6]: an uncalibrated pipeline reports improvements that evaporate under controlled re-evaluation. The protocol of Section IV is cheap relative to any evolution campaign (here, $\sim$70 extra placements) and catches both failure modes we encountered in practice—dead code paths and noise-fitting—before they reach a results table.

Consistency with concurrent work. Yao et al. [1] rank initialization $\gg$ optimizer $>$ preconditioner by both evolvability and gain, and report only 0.03–0.59% on their cleanest cases. Our schedule bound of $\sim$0.3% on standard-cell designs is quantitatively consistent with their ordering, measured for a component class they did not study and qualified with variance their study lacks.

VIII. Conclusion

We built a measurement-first harness for LLM-guided evolution of DREAMPlace schedules and ran a complete, noise-calibrated campaign of γ-schedule evolution on ISPD 2015 standard-cell designs. The campaign's headline is a bound, not a breakthrough: after 200 iterations, exactly one schedule survives paired multi-seed scrutiny, at $+0.315\% \pm 0.09\%$ HPWL versus the tuned default—while the single-seed ranking inverted, the apparent winner dissolving into seed noise. The result quantifies the headroom of a previously unexamined component class, corroborates component-importance orderings reported concurrently, and demonstrates a lightweight evaluation protocol—noise-floor measurement, liveness gating, paired multi-seed confirmation with a calibration row—that any stochastic-EDA search loop can adopt.

Acknowledgments

Experiments were carried out on an NVIDIA DGX Spark (GB10). The campaign's laboratory notebook, all candidate programs, raw logs, and the evaluation harness are available in the project repository.

References

[1] X. Yao, J. Jiang, Y. Zhao, P. Liao, Y. Lin, and B. Yu, "Evolution of optimization algorithms for global placement via large language models," 2025, arXiv:2504.17801.

[2] Y. Lin, S. Dhar, W. Li, H. Ren, B. Khailany, and D. Z. Pan, "DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement," in Proc. Design Automation Conference (DAC), 2019.

[3] J. Lu, P. Chen, C.-C. Chang, L. Sha, D. J.-H. Huang, C.-C. Teng, and C.-K. Cheng, "ePlace: Electrostatics-based placement using Fast Fourier Transform and Nesterov's method," ACM Trans. Design Automation of Electronic Systems, vol. 20, no. 2, 2015.

[4] A. Novikov et al., "AlphaEvolve: A coding agent for scientific and algorithmic discovery," 2025, google DeepMind technical report.

[5] C.-K. Cheng, A. B. Kahng, S. Kundu, Y. Wang, and Z. Wang, "Assessment of reinforcement learning for macro placement," in Proc. International Symposium on Physical Design (ISPD), 2023.

[6] I. L. Markov, "The false dawn: Reevaluating Google's reinforcement learning for chip macro placement," 2023, arXiv:2306.09633.

[7] "See it to place it: Evolving macro placements with vision-language models," 2026, arXiv:2603.28733.

[8] A. Mirhoseini, A. Goldie, M. Yazgan et al., "A graph placement methodology for fast chip design," Nature, vol. 594, pp. 207–212, 2021.

[9] "RoutePlacer: An end-to-end routability-aware placer with graph neural network," 2024, arXiv:2406.02651; Proc. ACM SIGKDD 2024.

[10] I. S. Bustany, D. Chinnery, J. R. Shinnerl, and V. Yutsis, "ISPD 2015 benchmarks with fence regions and routability constraints," in Proc. International Symposium on Physical Design (ISPD), 2015.